

Punto 5.12 — Buenas prácticas y anti-patrones en persistencia empresarial

Práctica

Ejercicio: Revisar el código del proyecto AceriaData para aplicar las buenas prácticas y eliminar los anti-patrones. Crear un caso de uso que documente las buenas prácticas y los anti-patrones. Aplicar las correcciones necesarias.

Contexto del proyecto: En el punto 5.11 se profundizó en el testing con EF Core, incluyendo el uso de InMemory y SQLite en memoria. En este punto se consolidan las buenas prácticas y los anti-patrones, cerrando el Módulo 5. Este punto es el cierre del curso.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el caso de uso de buenas prácticas y anti-patrones

Crear el archivo src/AceriaData.Application/UseCases/BuenasPracticasAntiPatronesUseCase.cs:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class BuenasPracticasAntiPatronesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public BuenasPracticasAntiPatronesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== BUENAS PRÁCTICAS Y ANTI-PATRONES ===");

        MostrarBuenasPracticasCicloDeVida();
        MostrarBuenasPracticasModelado();
        MostrarBuenasPracticasConsultas();
        MostrarBuenasPracticasEscritura();
        MostrarBuenasPracticasMigraciones();
        MostrarBuenasPracticasTesting();
        MostrarAntiPatronesHabituales();
    }

    private void MostrarBuenasPracticasCicloDeVida()
    {
        Console.WriteLine("\n--- Ciclo de vida del DbContext ---");
        Console.WriteLine("1. Crear el DbContext por unidad de trabajo.");
        Console.WriteLine("2. Liberar el DbContext con using.");
        Console.WriteLine("3. No compartir el DbContext entre hilos.");
        Console.WriteLine("4. No mantener el DbContext vivo durante toda la aplicación.");
    }

    private void MostrarBuenasPracticasModelado()
    {
        Console.WriteLine("\n--- Modelado y configuración ---");
        Console.WriteLine("1. Usar Fluent API para la configuración.");
        Console.WriteLine("2. Configurar claves, índices y restricciones explícitamente.");
    }
}
```

```

        Console.WriteLine("3. Configurar longitudes máximas y precisión decimal.");
        Console.WriteLine("4. Usar filtros globales para Soft Delete.");
    }

    private void MostrarBuenasPracticasConsultas()
    {
        Console.WriteLine("\n--- Consultas y carga de datos ---");
        Console.WriteLine("1. Usar proyecciones para reducir el volumen de datos.");
        Console.WriteLine("2. Usar AsNoTracking en consultas de solo lectura.");
        Console.WriteLine("3. Usar Include para cargar entidades relacionadas.");
        Console.WriteLine("4. Usar AsSplitQuery cuando se incluyen varias colecciones.");
        Console.WriteLine("5. Aplicar filtros y paginación en el servidor.");
        Console.WriteLine("6. Evitar funciones en Where.");
        Console.WriteLine("7. Evitar métodos personalizados en Where.");
    }

    private void MostrarBuenasPracticasEscritura()
    {
        Console.WriteLine("\n--- Escritura y transacciones ---");
        Console.WriteLine("1. Agrupar operaciones en una unidad de trabajo.");
        Console.WriteLine("2. Usar transacciones explícitas cuando sea necesario.");
        Console.WriteLine("3. Mantener las transacciones cortas.");
        Console.WriteLine("4. Gestionar los conflictos de concurrencia.");
    }

    private void MostrarBuenasPracticasMigraciones()
    {
        Console.WriteLine("\n--- Migraciones y despliegue ---");
        Console.WriteLine("1. Generar migraciones con nombres descriptivos.");
        Console.WriteLine("2. No modificar migraciones ya aplicadas.");
        Console.WriteLine("3. Usar scripts idempotentes en producción.");
        Console.WriteLine("4. Hacer copias de seguridad antes de aplicar.");
        Console.WriteLine("5. Preparar planes de reversión.");
    }

    private void MostrarBuenasPracticasTesting()
    {
        Console.WriteLine("\n--- Testing y diagnóstico ---");
        Console.WriteLine("1. Usar SQLite en memoria para tests realistas.");
        Console.WriteLine("2. Usar InMemory para tests rápidos.");
        Console.WriteLine("3. Configurar logging con ILogger o Serilog.");
        Console.WriteLine("4. Usar observadores de diagnóstico para detectar consultas lentas.");
    }

    private void MostrarAntiPatronesHabituales()
    {
        Console.WriteLine("\n--- Anti-patrones habituales ---");
        Console.WriteLine("1. DbContext estático compartido.");
        Console.WriteLine("2. Repositorio genérico excesivo.");
        Console.WriteLine("3. Exponer IQueryable.");
        Console.WriteLine("4. Problema N+1.");
        Console.WriteLine("5. Over-fetching.");
        Console.WriteLine("6. Carga Lazy sin control.");
        Console.WriteLine("7. Materialización prematura.");
        Console.WriteLine("8. Funciones en Where.");
        Console.WriteLine("9. Métodos personalizados en Where.");
        Console.WriteLine("10. Transacciones largas.");
        Console.WriteLine("11. Migraciones modificadas.");
        Console.WriteLine("12. Tests que siempre pasan.");
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class BuenasPracticasAntiPatronesUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public BuenasPracticasAntiPatronesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: `_unidad = unidad;` → asigna el parámetro al campo.

Línea 14: `public void Ejecutar()` → declara el método principal.

Línea 16: `Console.WriteLine("=== BUENAS PRÁCTICAS Y ANTI-PATRONES ===");` → muestra la cabecera.

Línea 18: `MostrarBuenasPracticasCicloDeVida();` → llama al método del ciclo de vida.

Línea 19: `MostrarBuenasPracticasModelado();` → llama al método de modelado.

Línea 20: `MostrarBuenasPracticasConsultas();` → llama al método de consultas.

Línea 21: `MostrarBuenasPracticasEscritura();` → llama al método de escritura.

Línea 22: `MostrarBuenasPracticasMigraciones();` → llama al método de migraciones.

Línea 23: `MostrarBuenasPracticasTesting();` → llama al método de testing.

Línea 24: `MostrarAntiPatronesHabituales();` → llama al método de anti-patrones.

Línea 27: `private void MostrarBuenasPracticasCicloDeVida()` → declara el método.

Línea 29: `Console.WriteLine("\n--- Ciclo de vida del DbContext ---");` → muestra la cabecera.

Línea 30: `Console.WriteLine("1. Crear el DbContext por unidad de trabajo.");` → describe la primera práctica.

Línea 31: `Console.WriteLine("2. Liberar el DbContext con using.");` → describe la segunda práctica.

Línea 32: `Console.WriteLine("3. No compartir el DbContext entre hilos.");` → describe la tercera práctica.

Línea 33: `Console.WriteLine("4. No mantener el DbContext vivo durante toda la aplicación.");` → describe la cuarta práctica.

Línea 36: `private void MostrarBuenasPracticasModelado()` → declara el método.

Línea 38: `Console.WriteLine("\n--- Modelado y configuración ---");` → muestra la cabecera.

Línea 39: `Console.WriteLine("1. Usar Fluent API para la configuración.");` → describe la primera práctica.

Línea 40: `Console.WriteLine("2. Configurar claves, índices y restricciones explícitamente.");` → describe la segunda práctica.

Línea 41: `Console.WriteLine("3. Configurar longitudes máximas y precisión decimal.");` → describe la tercera práctica.

Línea 42: `Console.WriteLine("4. Usar filtros globales para Soft Delete.");` → describe la cuarta práctica.

Línea 45: `private void MostrarBuenasPracticasConsultas()` → declara el método.

Línea 47: `Console.WriteLine("\n--- Consultas y carga de datos ---");` → muestra la cabecera.

Línea 48: `Console.WriteLine("1. Usar proyecciones para reducir el volumen de datos.");` → describe la primera práctica.

Línea 49: `Console.WriteLine("2. Usar AsNoTracking en consultas de solo lectura.");` → describe la segunda práctica.

Línea 50: `Console.WriteLine("3. Usar Include para cargar entidades relacionadas.");` → describe la tercera práctica.

Línea 51: `Console.WriteLine("4. Usar AsSplitQuery cuando se incluyen varias colecciones.");` → describe la cuarta práctica.

Línea 52: `Console.WriteLine("5. Aplicar filtros y paginación en el servidor.");` → describe la quinta práctica.

Línea 53: `Console.WriteLine("6. Evitar funciones en Where.");` → describe la sexta práctica.

Línea 54: `Console.WriteLine("7. Evitar métodos personalizados en Where.");` → describe la séptima práctica.

Línea 57: `private void MostrarBuenasPracticasEscritura()` → declara el método.

Línea 59: `Console.WriteLine("\n--- Escritura y transacciones ---");` → muestra la cabecera.

Línea 60: `Console.WriteLine("1. Agrupar operaciones en una unidad de trabajo.");` → describe la primera práctica.

Línea 61: `Console.WriteLine("2. Usar transacciones explícitas cuando sea necesario.");` → describe la segunda práctica.

Línea 62: `Console.WriteLine("3. Mantener las transacciones cortas.");` → describe la tercera práctica.

Línea 63: `Console.WriteLine("4. Gestionar los conflictos de concurrencia.");` → describe la cuarta práctica.

Línea 66: `private void MostrarBuenasPracticasMigraciones()` → declara el método.

Línea 68: `Console.WriteLine("\n--- Migraciones y despliegue ---");` → muestra la cabecera.

Línea 69: `Console.WriteLine("1. Generar migraciones con nombres descriptivos.");` → describe la primera práctica.

Línea 70: `Console.WriteLine("2. No modificar migraciones ya aplicadas.");` → describe la segunda práctica.

Línea 71: `Console.WriteLine("3. Usar scripts idempotentes en producción.");` → describe la tercera práctica.

Línea 72: `Console.WriteLine("4. Hacer copias de seguridad antes de aplicar.");` → describe la cuarta práctica.

Línea 73: `Console.WriteLine("5. Preparar planes de reversión.");` → describe la quinta práctica.

Línea 76: `private void MostrarBuenasPracticasTesting()` → declara el método.

Línea 78: `Console.WriteLine("\n--- Testing y diagnóstico ---");` → muestra la cabecera.

Línea 79: `Console.WriteLine("1. Usar SQLite en memoria para tests realistas.");` → describe la primera práctica.

Línea 80: `Console.WriteLine("2. Usar InMemory para tests rápidos.");` → describe la segunda práctica.

Línea 81: `Console.WriteLine("3. Configurar logging con ILogger o Serilog.");` → describe la tercera práctica.

Línea 82: `Console.WriteLine("4. Usar observadores de diagnóstico para detectar consultas lentas.");` → describe la cuarta práctica.

Línea 85: `private void MostrarAntiPatronesHabituales()` → declara el método.

Línea 87: `Console.WriteLine("\n--- Anti-patrones habituales ---");` → muestra la cabecera.

Línea 88: `Console.WriteLine("1. DbContext estático compartido.");` → describe el primer anti-patrón.

Línea 89: `Console.WriteLine("2. Repositorio genérico excesivo.");` → describe el segundo anti-patrón.

Línea 90: `Console.WriteLine("3. Exponer IQueryable.");` → describe el tercer anti-patrón.

Línea 91: `Console.WriteLine("4. Problema N+1.");` → describe el cuarto anti-patrón.

Línea 92: `Console.WriteLine("5. Over-fetching.");` → describe el quinto anti-patrón.

Línea 93: `Console.WriteLine("6. Carga Lazy sin control.");` → describe el sexto anti-patrón.

Línea 94: `Console.WriteLine("7. Materialización prematura.");` → describe el séptimo anti-patrón.

Línea 95: `Console.WriteLine("8. Funciones en Where.");` → describe el octavo anti-patrón.

Línea 96: `Console.WriteLine("9. Métodos personalizados en Where.");` → describe el noveno anti-patrón.

Línea 97: `Console.WriteLine("10. Transacciones largas.");` → describe el décimo anti-patrón.

Línea 98: `Console.WriteLine("11. Migraciones modificadas.");` → describe el undécimo anti-patrón.

Línea 99: `Console.WriteLine("12. Tests que siempre pasan.");` → describe el duodécimo anti-patrón.

Error común: si el caso de uso no documenta los anti-patrones, los desarrolladores pueden repetirlos. Se deben documentar todos.

Paso 3: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<BuenasPracticasAntiPatronesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

Paso 4: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<BuenasPracticasAntiPatronesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

Paso 5: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con las buenas prácticas y los anti-patrones.

Paso 6: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== BUENAS PRÁCTICAS Y ANTI-PATRONES ===

--- Ciclo de vida del DbContext ---
1. Crear el DbContext por unidad de trabajo.
2. Liberar el DbContext con using.
3. No compartir el DbContext entre hilos.
4. No mantener el DbContext vivo durante toda la aplicación.

--- Modelado y configuración ---
1. Usar Fluent API para la configuración.
2. Configurar claves, índices y restricciones explícitamente.
3. Configurar longitudes máximas y precisión decimal.
```

4. Usar filtros globales para Soft Delete.

--- Consultas y carga de datos ---

1. Usar proyecciones para reducir el volumen de datos.
2. Usar AsNoTracking en consultas de solo lectura.
3. Usar Include para cargar entidades relacionadas.
4. Usar AsSplitQuery cuando se incluyen varias colecciones.
5. Aplicar filtros y paginación en el servidor.
6. Evitar funciones en Where.
7. Evitar métodos personalizados en Where.

--- Escritura y transacciones ---

1. Agrupar operaciones en una unidad de trabajo.
2. Usar transacciones explícitas cuando sea necesario.
3. Mantener las transacciones cortas.
4. Gestionar los conflictos de concurrencia.

--- Migraciones y despliegue ---

1. Generar migraciones con nombres descriptivos.
2. No modificar migraciones ya aplicadas.
3. Usar scripts idempotentes en producción.
4. Hacer copias de seguridad antes de aplicar.
5. Preparar planes de reversión.

--- Testing y diagnóstico ---

1. Usar SQLite en memoria para tests realistas.
2. Usar InMemory para tests rápidos.
3. Configurar logging con ILogger o Serilog.
4. Usar observadores de diagnóstico para detectar consultas lentas.

--- Anti-patrones habituales ---

1. DbContext estático compartido.
 2. Repositorio genérico excesivo.
 3. Exponer IQueryable.
 4. Problema N+1.
 5. Over-fetching.
 6. Carga Lazy sin control.
 7. Materialización prematura.
 8. Funciones en Where.
 9. Métodos personalizados en Where.
 10. Transacciones largas.
 11. Migraciones modificadas.
 12. Tests que siempre pasan.
- La salida muestra las buenas prácticas y los anti-patrones agrupados por categoría.

Observaciones: las buenas prácticas cubren todos los aspectos del acceso a datos. Los anti-patrones son los errores más habituales que se deben evitar.

Paso 7: Diagnosticar un error común

Modificar el caso de uso para usar un DbContext estático compartido:

```
public class BuenasPracticasAntiPatronesUseCase
{
    private static readonly AcriaDbContext _contextCompartido = new AcriaDbContext();
    // ...
}
```

Resultado esperado: el código compila pero el DbContext compartido acumula entidades en el Change Tracker y consume memoria. Además, no es seguro para subprocesos.

Solución: usar la unidad de trabajo inyectada por el contenedor.

```
public class BuenasPracticasAntiPatronesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public BuenasPracticasAntiPatronesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }
}
```

Resultado esperado con la solución: el DbContext se crea por ámbito y se libera al final.

Errores comunes del ejercicio

Error Causa Solución DbContext estático Se comparte entre toda la aplicación Usar contexto por unidad de trabajo Repositorio expone IQueryable Rompe la abstracción Devolver listas N+1 No se usa Include Usar Include Over-fetching Se cargan entidades completas Usar proyecciones Carga Lazy sin control Se accede a propiedades de navegación en bucle Usar carga Eager Materialización prematura Se llama a ToList antes de filtrar Filtrar antes de materializar Funciones en Where Se aplican funciones sobre columnas Comparar directamente Métodos personalizados en Where No se traducen a SQL Reescribir con expresiones traducibles Transacciones largas Se mantienen abiertas durante mucho tiempo Mantener cortas Migraciones modificadas Se modifican migraciones aplicadas Generar una nueva migración Tests que siempre pasan No verifican nada Verificar el comportamiento real

Reto resuelto: Auditoría de buenas prácticas en el repositorio

Reto: Crear un método en el repositorio que audite las consultas del proyecto y reporte cuáles usan buenas prácticas y cuáles no. El método debe devolver una lista de hallazgos.

Solución paso a paso:

Paso 1: Crear el DTO HallazgoAuditoriaDto:

```
namespace AceriaData.Application.Dtos;

public class HallazgoAuditoriaDto
{
    public string Metodo { get; set; } = string.Empty;
    public string Practica { get; set; } = string.Empty;
    public string Recomendacion { get; set; } = string.Empty;
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class HallazgoAuditoriaDto → declara el DTO.

Línea 5: public string Metodo { get; set; } = string.Empty; → nombre del método.

Línea 6: public string Practica { get; set; } = string.Empty; → práctica evaluada.

Línea 7: public string Recomendacion { get; set; } = string.Empty; → recomendación.

Paso 2: Añadir el método a la interfaz IOrdenRepositorio:

```
List<HallazgoAuditoriaDto> AuditarBuenasPracticas();
```

Línea 1: List AuditarBuenasPracticas(); → declara el método.

Paso 3: Implementar el método en OrdenRepositorio:

```
public List<HallazgoAuditoriaDto> AuditarBuenasPracticas()
{
    return new List<HallazgoAuditoriaDto>
    {
        new HallazgoAuditoriaDto
        {
            Metodo = "ObtenerResumenes",
            Practica = "Usa AsNoTracking y proyección",
            Recomendacion = "Correcto"
        },
        new HallazgoAuditoriaDto
        {
            Metodo = "ObtenerConPlanchasInclude",
            Practica = "Usa Include y AsNoTracking",
            Recomendacion = "Correcto"
        },
        new HallazgoAuditoriaDto
        {
            Metodo = "ObtenerConPlanchasYDetalleSplitQuery",
            Practica = "Usa AsSplitQuery",
            Recomendacion = "Correcto"
        },
        new HallazgoAuditoriaDto
        {
            Metodo = "ObtenerPorIdParaActualizar",
            Practica = "Usa tracking para modificar",
            Recomendacion = "Correcto"
        }
    };
}
```

Línea 1: public List AuditarBuenasPracticas() → declara el método.

Línea 3: return new List → crea la lista.

Línea 5: new HallazgoAuditoriaDto → crea el primer hallazgo.

Línea 7: Metodo = "ObtenerResumenes", → asigna el método.

Línea 8: Practica = "Usa AsNoTracking y proyección", → asigna la práctica.

Línea 9: Recomendacion = "Correcto" → asigna la recomendación.

Línea 11: new HallazgoAuditoriaDto → crea el segundo hallazgo.

Línea 17: new HallazgoAuditoriaDto → crea el tercer hallazgo.

Línea 23: new HallazgoAuditoriaDto → crea el cuarto hallazgo.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarAuditoria()
{
    Console.WriteLine("\n--- Auditoría de buenas prácticas ---");

    var hallazgos = _unidad Ordenes.AuditarBuenasPracticas();
    foreach (var hallazgo in hallazgos)
    {
        Console.WriteLine($"Método: {hallazgo.Metodo} | Práctica: {hallazgo.Practica} | Recomendación: {hallazgo.Recomendacion}");
    }
}
```

Línea 1: private void DemostrarAuditoria() → declara el método.

Línea 3: Console.WriteLine("\n--- Auditoría de buenas prácticas ---"); → muestra la cabecera.

Línea 5: `var hallazgos = _unidad Ordenes.AuditarBuenasPracticas();` → llama al método del repositorio.

Línea 6: `foreach (var hallazgo in hallazgos)` → itera sobre los hallazgos.

Línea 8: `Console.WriteLine($"Método: {hallazgo.Metodo} | Práctica: {hallazgo.Practica} | Recomendación: {hallazgo.Recomendacion}");` → muestra el hallazgo.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarAuditoria();
```

Paso 6: Ejecutar dotnet run y verificar que los hallazgos se muestran.

Resultado esperado: se muestra la auditoría de buenas prácticas con los métodos y sus recomendaciones.

Ampliación técnica obligatoria — refactorización ejecutable de N+1

La lista de buenas prácticas no basta. El cierre del módulo debe demostrar al menos un anti-patrón completo, su evidencia y dos refactorizaciones.

Paso 8: Reproducir un N+1 intencionado

Carga las órdenes y ejecuta una consulta adicional por orden para contar sus planchas. Reinicia el interceptor antes del escenario y conserva el resultado funcional para compararlo después.

```
SqlCommandCounterInterceptor.Instance.Reset();

var ordenes = context.OrdenesFabricacion
    .AsNoTracking()
    .OrderBy(o => o.Id)
    .ToList();

var resultadoN1 = ordenes.Select(o => new
{
    o.NumeroOrden,
    TotalPlanchas = context.PlanchasAcero.Count(p => p.OrdenId == o.Id)
}).ToList();

var comandosN1 = SqlCommandCounterInterceptor.Instance.Count;
```

Paso 9: Primera corrección con carga anticipada cuando necesitas el grafo

```
SqlCommandCounterInterceptor.Instance.Reset();

var conInclude = context.OrdenesFabricacion
    .AsNoTracking()
    .Include(o => o.Planchas)
    .OrderBy(o => o.Id)
    .ToList()
    .Select(o => new
    {
        o.NumeroOrden,
        TotalPlanchas = o.Planchas.Count
    })
    .ToList();

var comandosInclude = SqlCommandCounterInterceptor.Instance.Count;
```

`Include` es una posible corrección cuando el grafo completo es necesario; no se presenta como solución universal.

Paso 10: Segunda mejora con proyección

```
SqlCommandCounterInterceptor.Instance.Reset();

var proyectado = context.OrdenesFabricacion
    .AsNoTracking()
    .OrderBy(o => o.Id)
    .Select(o => new
    {
        o.NumeroOrden,
        TotalPlanchas = o.Planchas.Count()
    })
    .ToList();

var comandosProyeccion = SqlCommandCounterInterceptor.Instance.Count;
```

La proyección evita materializar el grafo completo cuando solo se necesita el total.

Paso 11: Verificar equivalencia funcional y evidencia técnica

```
Assert.Equal(
    resultadoN1.Select(x => (x.NumeroOrden, x.TotalPlanchas)),
    conInclude.Select(x => (x.NumeroOrden, x.TotalPlanchas)));

Assert.Equal(
    resultadoN1.Select(x => (x.NumeroOrden, x.TotalPlanchas)),
    proyectado.Select(x => (x.NumeroOrden, x.TotalPlanchas)));
```

Registra para cada versión:

- comandos SQL / roundtrips observados;
- entidades rastreadas;
- forma de las columnas materializadas a partir del SQL real;
- resultado funcional.

No fijas porcentajes de mejora universales. La conclusión procede de la evidencia de esa ejecución.

Paso 12: Matriz final de anti-patrones y trade-offs

Anti-patrón	Síntoma	Consecuencia	Evidencia	Refactor	Trade-off
N+1	una consulta inicial + consultas por fila	más roundtrips	interceptor / logs SQL	Include o proyección según necesidad	Include materializa grafo; proyección cambia la forma devuelta
Over-fetching	columnas/ entidades no usadas	más datos materializados	SQL y forma del resultado	proyección	requiere DTO/ forma específica
Tracking innecesario	ChangeTracker crece en lectura	coste de tracking	ChangeTracker.Entries()	AsNoTracking	no usarlo cuando se editarán esas entidades
SplitQuery automático	varias consultas sin justificación	más roundtrips	logs/interceptor	elegir Single/ Split por escenario	Split reduce explosión cartesiana pero aumenta roundtrips

Anti-patrón	Síntoma	Consecuencia	Evidencia	Refactor	Trade-off
Repository como dogma	abstracción sin necesidad	mantenimiento adicional	arquitectura/cobertura	usarlo cuando aporta aislamiento/ testabilidad	DbContext ya ofrece UoW/ Repository-like capabilities

Paso 13: Reglas que deben quedar matizadas

- Data Annotations no son un anti-patrón por sí mismas; Fluent API ofrece más capacidad y centralización, pero la elección depende del modelo.
- `Include` no es siempre la mejor corrección para N+1; una proyección puede ser más adecuada.
- `AsSplitQuery` no se aplica automáticamente: reduce determinadas explosiones cartesianas a cambio de más roundtrips y consideraciones de consistencia.
- Un método `.NET` no traducible dentro de un `Where` de EF Core 8 normalmente produce un error de traducción salvo que se introduzca explícitamente una frontera cliente.
- Aplicar funciones a columnas puede perjudicar la sargabilidad, pero el uso real de índices debe comprobarse con el proveedor, el índice, la collation y el plan de ejecución.

Analogía final

Las buenas prácticas y los anti-patrones en una acería son como el manual de procedimientos de la planta. Las buenas prácticas son los procedimientos que han demostrado ser seguros y eficientes. Los anti-patrones son los procedimientos que parecen correctos pero que provocan accidentes o pérdidas. El manual se actualiza con la experiencia y se consulta antes de cada operación. Las buenas prácticas del ciclo de vida del DbContext son como las normas de uso de las herramientas: se usan y se devuelven al taller. Las buenas prácticas de modelado son como los planos bien dibujados: se entienden y se mantienen. Las buenas prácticas de consultas son como las órdenes de búsqueda eficientes: se pide solo lo necesario. Las buenas prácticas de escritura son como los lotes de producción: se agrupan y se confirman. Las buenas prácticas de migraciones son como las reformas controladas: se planifican y se revierten si fallan. Las buenas prácticas de testing son como los controles de calidad: se ejecutan antes de enviar. Los anti-patrones son los errores que se deben evitar. Así funcionan las buenas prácticas y los anti-patrones en EF Core: se documentan, se aplican y se revisan.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado el caso de uso BuenasPracticasAntiPatronesUseCase.

Registrado el caso de uso en el contenedor.

Ejecutado el caso de uso y verificado las buenas prácticas y los anti-patrones.

Diagnosticado el error del DbContext estático.

Creado el DTO HallazgoAuditoriaDto y el método AuditarBuenasPracticas.

Resumen del estado del proyecto AceriaData al final del Módulo 5 Al final del Módulo 5, el proyecto AceriaData tiene:

La arquitectura limpia configurada en cuatro proyectos: AceriaData.Domain, AceriaData.Application, AceriaData.Infrastructure y AceriaData.Console.

El dominio con las entidades OrdenFabricacion, PlanchaAcero, Aleacion, EstadoOrden, DetalleOrden, CertificadoCalidad y OrdenAleacion.

La aplicación con las interfaces IRepository, IOrdenRepository, IPlanchaRepository, IAleacionRepository, IDetalleOrdenRepository e IUnidadDeTrabajo, los casos de uso de todos los módulos y los DTOs.

La infraestructura con el `AceriaDbContext`, las configuraciones de Fluent API, las implementaciones de repositorios, la unidad de trabajo, las consultas compiladas y el observador de diagnóstico.

La consola con el método `Main` y la configuración del contenedor de dependencias.

El modelo de datos completo con relaciones uno a muchos, uno a uno y muchos a muchos.

Las claves primarias, alternativas y compuestas configuradas.

Los índices y las restricciones configurados.

El `Soft Delete` implementado con filtros globales.

La arquitectura limpia aplicada con la regla de dependencia respetada.

Las consultas LINQ con filtros, ordenaciones, proyecciones, agregaciones, agrupaciones, joins, carga Eager, carga Lazy, carga Explicit y composición de consultas.

Las buenas prácticas de acceso a datos aplicadas.

La optimización del rendimiento aplicada: análisis del SQL, Tracking vs No Tracking, resolución de identidad, solución al N+1, over-fetching, consultas ineficientes, Split Queries, Compiled Queries, paginación eficiente, diagnóstico y checklist de rendimiento.

La persistencia empresarial aplicada: concurrencia optimista, tokens de concurrencia, resolución de conflictos, transacciones, transacciones ambientales, migraciones en producción, migraciones idempotentes, migraciones en equipos, patrón Repositorio y unidad de trabajo, logging y diagnóstico, testing con EF Core y buenas prácticas.
